

Nexus ERP Fase 5

Para que o seu sistema pareça um ERP profissional, ele precisa de **reatividade**: você salva um dado e ele aparece na tela instantaneamente.

Aqui está a atualização completa do **Backend** e do **Frontend** para incluir a funcionalidade de **Listar Vendas** (puxando direto do PostgreSQL real).

1. Atualização do Backend (`main.py`)

Adicione este novo endpoint `GET /sales/list` para buscar as vendas no banco de dados.

```
` `` python
```

```
# ... (mantenha o código anterior de conexão e modelos) ...
```

```
@app.get("/sales/list")
```

```
def list_sales(db: Session = Depends(get_db)):
```

```
    # Busca todas as vendas ordenadas pela data mais recente
```

```
    vendas = db.query(Sale).order_by(Sale.created_at.desc()).all()
```

```
    return vendas
```

```
# ... (mantenha o restante dos endpoints) ...
```

```
` ``
```

2. Atualização do Frontend (`index.html`)

Agora vamos dar “inteligência” ao JavaScript para ele desenhar a tabela sempre que a página abrir ou uma venda nova for feita.

```html

<!-- Mantenha o HTML e CSS anterior, mude apenas a parte do <script> →

<script>

Const API\_URL = <http://localhost:8000>;

// 1. Função para carregar as vendas do PostgreSQL

Async function carregarVendas() {

Try {

Const response = await fetch(`\${API\_URL}/sales/list`);

Const vendas = await response.json();

Const corpoTabela = document.querySelector("#tabelaVendas tbody");

corpoTabela.innerHTML = ""; // Limpa a tabela antes de preencher

vendas.forEach(venda => {

const dataFormatada = new Date(venda.created\_at).toLocaleString('pt-BR');

const linha = `

<tr>

<td>\${venda.id.substring(0, 8)}...</td>

<td>\${venda.customer}</td>

<td>R\$ \${venda.total.toFixed(2)}</td>

</tr>

`;

corpoTabela.innerHTML += linha;

});

} catch (error) {

```
 Console.error("Erro ao carregar vendas:", error);
 }
}
```

// 2. Função para salvar a venda

```
Async function salvarVenda() {
 Const customer = document.getElementById('customer').value;
 Const value = document.getElementById('value').value;

 If (!customer || !value) {
 Alert("Por favor, preencha todos os campos.");
 Return;
 }
}
```

```
 Const response = await
 fetch(`${API_URL}/sales/new?customer=${customer}&value=${value}`, {
 Method: 'POST'
 });
```

```
 If (response.ok) {
 Alert("Venda gravada no PostgreSQL!");
 Document.getElementById('customer').value = '';
 Document.getElementById('value').value = '';
 }
```

// ATUALIZAÇÃO AUTOMÁTICA DA LISTA

```
 carregarVendas();
} else {
 Alert("Erro ao gravar venda.");
}
```

```
}
}
```

```
// 3. Executa ao abrir a página (Praticidade!)
```

```
Window.onload = carregarVendas;
```

```
</script>
```

```
...
```

---

### O que você tem agora (O Ciclo Completo):

1. **Entrada:** O usuário digita os dados.
2. **Processamento:** O Python valida, gera um ID (UUID) e grava no PostgreSQL.
3. **Persistência:** O banco confirma a gravação segura.
4. **Saída:** O sistema busca a lista atualizada do banco e redesenha a tabela para o usuário.

### Como testar:

1. Certifique-se de que o **Docker** (`nexus_db`) está ligado.
2. Rode o backend: `uvicorn main:app --reload`.
3. Abra o `index.html`.
4. Insira uma venda. Você verá que ela aparece na tabela **automaticamente**, sem precisar dar F5 na página.

---

### Próximo Passo Sugerido:

Agora que você domina o fluxo de **Vendas**, o que você gostaria de adicionar?

1. **Filtro de Busca:** Uma barra de pesquisa para filtrar clientes ou valores na tabela.
2. **Módulo de Compras (Entrada):** Criar uma tela similar para cadastrar o que a empresa compra.
3. **Dashboard de Lucro:** Uma pequena caixa no topo que soma o total de todas as vendas registradas no banco (Rentabilidade).

O que faz mais sentido para o seu aprendizado ou projeto agora?